# UCLA Anderson Math Bootcamp

Charles Rambo

UCLA Anderson

2026

# Table of Contents

# About Me

# About Me

**Email:** charles.tutoring@gmail.com

**LinkedIn:** `linkedin.com/in/charlesrambo`

Please email me your questions at my gmail account. Also, please feel free to connect with me on LinkedIn!

# Education

- MFE from UCLA in 2020
- BA in Mathematics from UC Berkeley in 2009
- CFA Charterholder and FRM Certified

# Experience

**State Street**

*Quantitative Analyst, AVP (Counterparty Credit Risk)* 2 months

- Focus on the modeling and assessment of counterparty credit exposures

*Quantitative Analyst, AVP (Model Risk Management)* 1 year

- Used quantitative methods to verify the integrity and performance of credit, liquidity, and macroeconomic models

# Experience

**SSI Investment Management**

*Portfolio Research Analyst* 4 years

- Firm specializes in convertible securities
- I did valuation, credit, equity, and portfolio construction

**Rambo Tutoring (Self-Employed)**

*Math Tutor and Author* 10 years

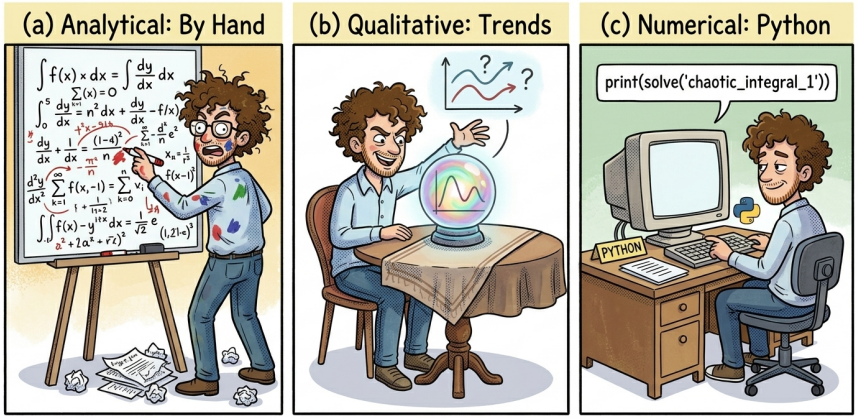
- Did tutoring for precalculus, calculus, linear algebra, differential equations, statistics, probability, and more!
- Wrote study material as well! Check out my stuff on Amazon.com!

# How the Course Works

# How the Course Works

- Videos avalable on BruinLearn
- Notes and quizzes posted on BruinLearn
- Grade of 65% or better to pass
- No exams

# Approaches to Solve Math Problems




- **Analytical:** Solve problems "by hand" to find an exact algebraic expression or a closed-form solution without the use of a computer.
- **Qualitative:** Analyze the general properties and trends of a system, such as stability or long-term limits, rather than calculating specific values.
- **Numerical:** Use Python to approximate values when an exact analytical solution is either impossible or impractical to find by hand.

# Tentative Course Outline

| Unit | Description | Sessions |
|---|---|---|
| 1 | Calculus | July 7–21 |
| 2 | Linear algebra and multivariable calculus | July 23–30 |
| 3 | Combinatorics, probability, and statistics | August 4–August 13 |
| 4 | Efficient frontier, PCA, and stochastic calculus | August 18–20 |

# References

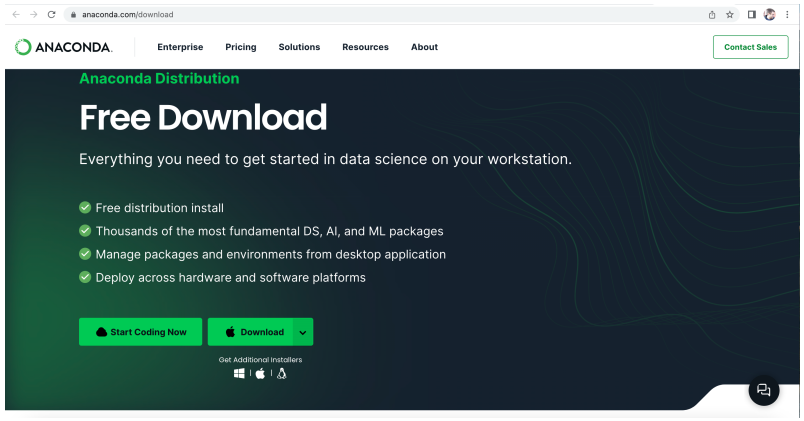
This is an incomplete list of references used to create the notes for this course. You do not need to purchase any of the books listed.


- James Stewart, *Calculus*, Brooks/Cole, 3rd ed., 1995.
- Walter Rudin, *Principles of Mathematical Analysis*, McGraw-Hill, 1976.
- Charles Pugh, *Real Mathematical Analysis*, Springer, 2002
- Serge Lang, *Linear Algebra*, Springer, 3rd ed., 1987.
- Steven Roman, *Advanced Linear Algebra*, Springer, 2nd ed., 2005.
- Morris DeGroot and Mark Schervish, *Probability and Statistics*, Pearson, 4th ed., 2013.
- Marcos Lopez de Prado, *Machine Learning for Asset Managers*, Cambridge University Press, 2020.
- Martin Haugh, *A Brief Introduction to Stochastic Calculus*, ⟨`https://www.columbia.edu/~mh2078/FoundationsFE/IntroStochCalc.pdf`⟩, Columbia University, 2016.

# Python

# Python Installation

- We'll be using Python to obtain numerical solutions.
- If you haven't used Python before, I suggest downloading Anaconda at `https://www.anaconda.com/download`

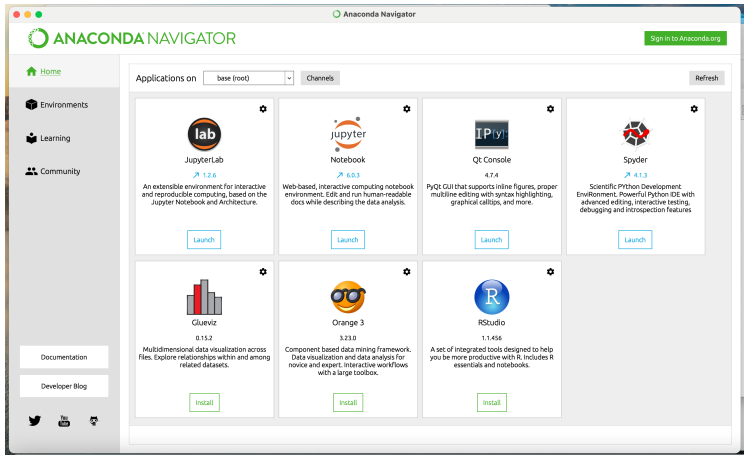


# Anaconda

After you've installed and opened Anaconda, a screen like this will appear.

You have several choices for IDEs. Spyder is useful for data analysis.

Jupyter Notebook is popular for explanatory work, so students and teachers tend to use it.



# Packages

In this class we'll be using the modules below.

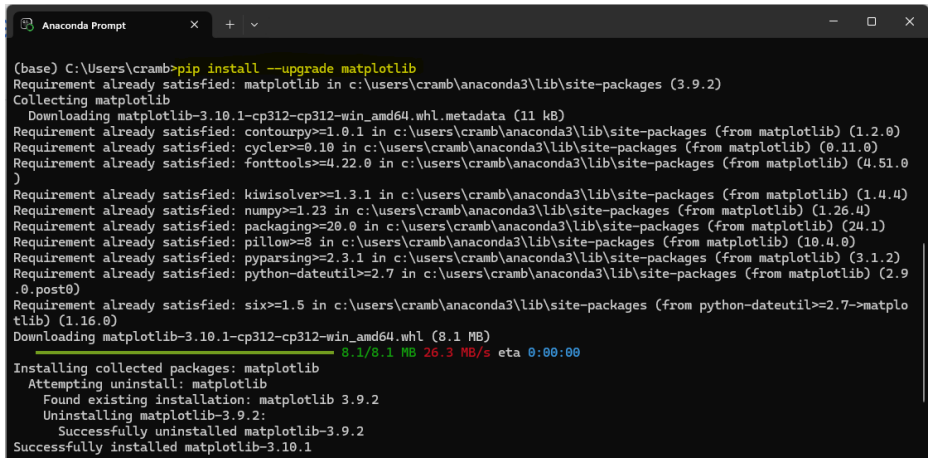
- *NumPy*: Useful for mathematical calculations.
- *Pandas*: Has data frame data type—it allows you to hold data in a format sort of like a spreadsheet.
- *Matplotlib:* Useful for graphing. Somewhat quirky syntax but very popular nonetheless.
- *SciPy*: Useful for scientific computing. It has more advanced math and statistics functions than NumPy.

# Package Installation

To install or upgrade a module, go into Anaconda Prompt (Windows) or Terminal (Mac/Linux), and type

```
pip install ...
```

In this example, I'm upgrading matplotlib.



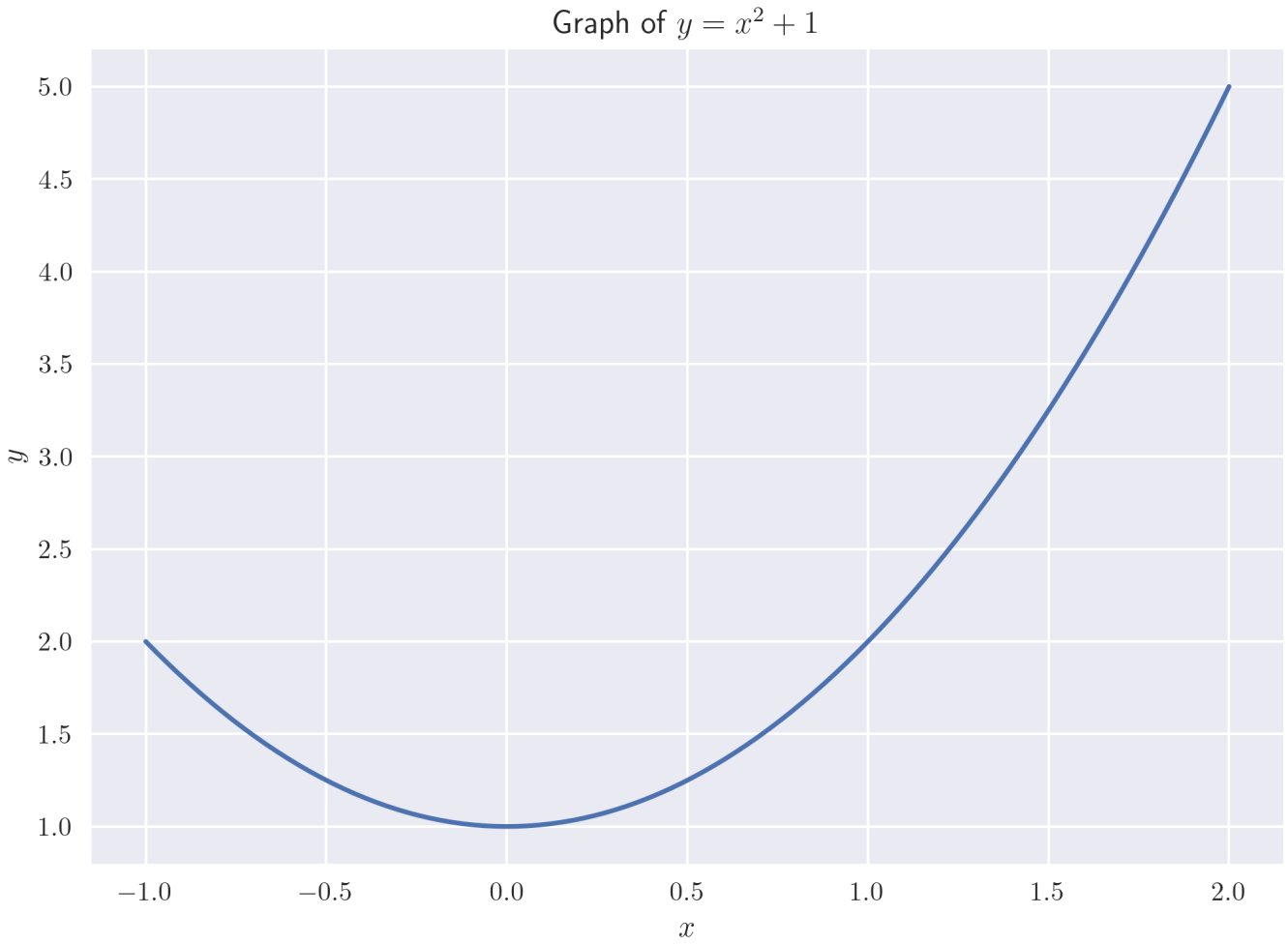
# Python Graphing Example

## Example

Let $f(x) = x^2 + 1$. Use Python to graph $f$ on the domain $[-1, 2]$.

```
# Import modules
import numpy as np
import matplotlib.pyplot as plt

# These three steps add style

# Use LaTex
plt.rcParams['text.usetex'] = True

# Increase image resolution
plt.rcParams['figure.dpi'] = 300

# Use Seaborn style
plt.style.use('seaborn-v0_8')

# Ready to do the math part

# Define f
def f(x):
    return x**2 + 1

# Another option is to use a lambda
    function
# f = lambda x: x**2 + 1

# Get 100 x-values in [-1, 2]
x_vals = np.linspace(-1, 2, 100)

# Use list comprehension to get y-values
y_vals = [f(x) for x in x_vals]

# Automatically vectorized so this works
    too
# y_vals = f(x_vals)

# Generate the plot
plt.plot(x_vals, y_vals)

# Label the x-axis
plt.xlabel('$x$')

# Label the y-axis
plt.ylabel('$y$')

# Give the graph a title
plt.title('Graph of $y = x^2 + 1$')

# Save the figure
plt.savefig(path + 'ex0-1')

# Display the plot
plt.show()
```

# Python Graphing Result



# Python Optimization Example

**Example**

Use Python to find the minimum of $f(x) = x^2 + 1$ on the interval $[-1, 2]$.

**Solution.** From the graph on the previous page, we know that the minimum is $y = 1$ which occurs when $x = 0$. But let's use Python to verify this. Suppose the code above is still in our local environment.

# Python Optimization Example

```
# Import minimize from scipy
from scipy.optimize import minimize

# Minimize function; set bounds equal to the domain
minimize(f, x0 = [1], bounds = [(-1, 2)])
```

The output is shown below.

```
 message: CONVERGENCE: NORM OF PROJECTED GRADIENT <= PGTOL
 success: True
  status: 0
     fun: 1.0
       x: [-8.412e-09]
     nit: 2
     jac: [ 0.000e+00]
    nfev: 8
    njev: 4
hess_inv: <1x1 LbfgsInvHessProduct with dtype=float64>
```

# LaTeX in Python

As you may have seen, I've enabled LaTeX rendering for most `matplotlib` graphs in this course. While `matplotlib` includes a native math-mode, LaTeX provides superior typesetting quality for quantitative work. While LaTeX integration is optional, it is encouraged and is required to execute my code snippets without modification. Please refer to the setup guide in `intro_code_snippets.ipynb` on BruinLearn to configure your environment.